

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN
Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



GUÍA DE INSTALACIÓN Y DESPLIEGUE: SISTEMA DISTRIBUIDO RPC (FLASK + RABBITMQ)

Docente: Jesús Martín Silva Fernández

Curso: Sistemas Distribuidos

Fecha: 06 de mayo de 2026

Integrantes:

Larico Rodriguez, Bryan Fernando
Maldonado Vilca, Victor Gonzalo
Quispe Huaman, Rodrigo Ferdinand
Salas Aguilar, Juan Victor

Arequipa - Perú

Índice

1. Descripción del Sistema	2
2. Arquitectura del Sistema	2
3. Requisitos Previos	3
4. Tecnologías Utilizadas	3
5. Instalación y Configuración Local	3
5.1. Obtención del código	3
5.2. Entorno de desarrollo	3
6. Configuración de RabbitMQ (CloudAMQP)	4
7. Despliegue en Render.com	4
7.1. Servicio 1: Web Service (Flask)	4
7.2. Servicio 2: Worker	4
8. Solución de Problemas	4
9. Conclusiones	4
10. Referencias	5

1. Descripción del Sistema

Este proyecto consiste en la implementación del patrón **Remote Procedure Call (RPC)** utilizando **Flask** como cliente/servidor HTTP y **RabbitMQ** (a través de CloudAMQP) como broker de mensajes. El sistema está diseñado para ser desplegado en **Render.com** mediante dos componentes independientes que se comunican exclusivamente a través de colas. El patrón RPC permite que un programa invoque procedimientos remotos como si fueran funciones locales. En este proyecto, RabbitMQ actúa como intermediario de mensajería, permitiendo desacoplar el cliente y el worker para mejorar escalabilidad y tolerancia a fallos.

2. Arquitectura del Sistema

El flujo de datos sigue un modelo asíncrono donde el broker actúa como intermediario:

1. El usuario realiza una petición al **Servidor Flask**.
2. Flask publica la tarea en la cola `rpc_queue` de **CloudAMQP**.
3. El **Worker** consume el mensaje, procesa la información (conversión a mayúsculas) y devuelve el resultado.
4. El cliente recupera el resultado y lo muestra al usuario.

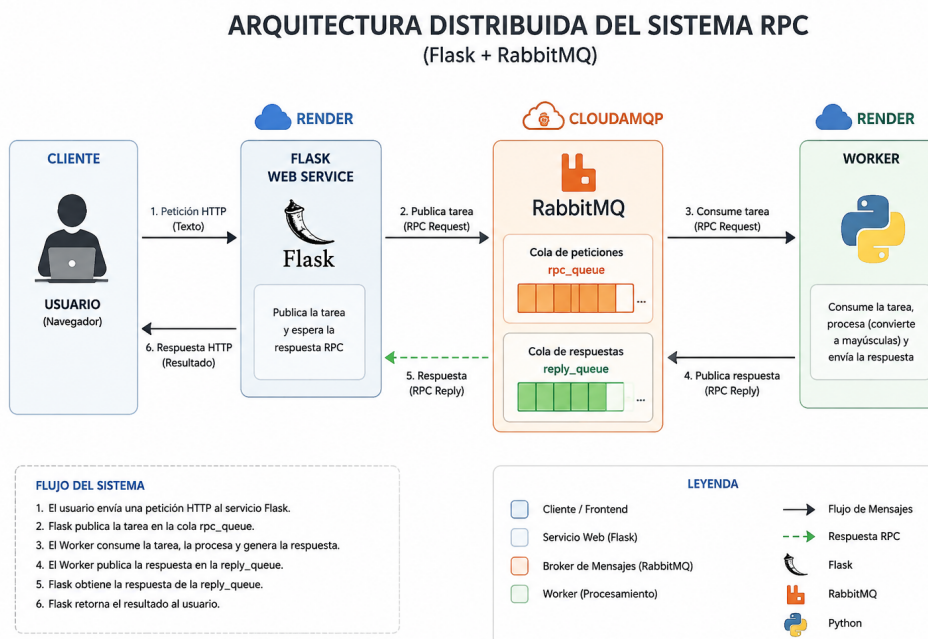


Figura 1: Arquitectura distribuida del sistema RPC

3. Requisitos Previos

Para la correcta ejecución del sistema, asegúrese de contar con:

- Python 3.9 o superior.
- Una cuenta activa en [Render.com](https://render.com).
- Una instancia gratuita (Little Lemur) en [CloudAMQP](https://cloudamqp.com).
- Herramienta Git instalada.

4. Tecnologías Utilizadas

- Python 3.11
- Flask
- RabbitMQ
- CloudAMQP
- Render.com
- Gunicorn
- AMQPStorm

5. Instalación y Configuración Local

5.1. Obtención del código

```
1 git clone https://github.com/Victor-Gonzalo-Maldonado-Vilca/  
  Sistemas_Distribuidos_RPC.git  
2 cd Sistemas_Distribuidos_RPC
```

5.2. Entorno de desarrollo

Cree un entorno virtual para aislar las dependencias:

```
1 python -m venv venv  
2 # Activar (Linux/Mac)  
3 source venv/bin/activate  
4 # Activar (Windows)  
5 venv\Scripts\activate
```

Instale los paquetes requeridos:

```
1 pip install -r requirements.txt
```

6. Configuración de RabbitMQ (CloudAMQP)

1. Acceda a su panel de CloudAMQP.
2. Copie la **AMQP URL**.
3. Reemplace el valor de la variable `URL_NUBE` en los archivos `worker.py` y `amqpstorm_threaded_rpc.py`.

7. Despliegue en Render.com

Para el despliegue en la nube, es crítico configurar dos servicios distintos para permitir el funcionamiento distribuido.

7.1. Servicio 1: Web Service (Flask)

- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `gunicorn amqpstorm_threaded_rpc_client:app`

7.2. Servicio 2: Worker

Debido a las limitaciones del plan gratuito, se despliega como un **Web Service** configurado para escuchar en el puerto 10000 (servidor dummy) para evitar que Render detenga el proceso.

- **Start Command:** `python worker.py`

8. Solución de Problemas

- **Timeout:** Si recibe un error de tiempo agotado, verifique que el servicio del Worker esté en estado **Live**. Los servicios gratuitos de Render suelen "dormirse" tras 15 minutos de inactividad.
- **Error de Conexión:** Asegúrese de que la URL de CloudAMQP sea la correcta y que no incluya caracteres extraños.

Los servicios gratuitos de Render entran en estado de suspensión tras 15 minutos de inactividad.

9. Conclusiones

- Se implementó correctamente un sistema distribuido basado en RPC.
- RabbitMQ permitió desacoplar el procesamiento entre cliente y worker.
- El despliegue en Render demostró la viabilidad de arquitecturas distribuidas en la nube.
- El sistema puede ampliarse fácilmente agregando múltiples workers.

10. Referencias

- RabbitMQ Documentation: <https://www.rabbitmq.com/>
- Flask Documentation: <https://flask.palletsprojects.com/>
- Render Documentation: <https://render.com/docs>
- CloudAMQP Documentation: <https://www.cloudamqp.com/docs/>